Integración de Sistemas

Semana 7. Esquemas, compatibilidad y gestión de contratos

Sistemas de Información — PUCE
Docente: Damián Nicolalde

Semana 7. Esquemas, compatibilidad y gestión de contratos

Objetivo: Comprender y aplicar los mecanismos formales que garantizan que dos sistemas construidos por equipos distintos, en momentos distintos, puedan seguir entendiéndose a medida que evolucionan: contratos REST vs. gRPC, los enfoques schema-first y code-first, los lenguajes de esquema JSON Schema/Avro/Protobuf, el rol de un Schema Registry, y las reglas de compatibilidad hacia atrás.

Resultado de aprendizaje asociado: Implementar un ecosistema de microservicios expuestos mediante APIs REST y gestionados con API Gateway y un bus de eventos, incorporando contratos, pruebas, observabilidad y seguridad para asegurar escalabilidad y desempeño.

Introducción

Desde la Semana 1 este curso repite, bajo distintos disfraces, la misma idea: un contrato explícito es lo que permite que dos sistemas se entiendan sin tener que llamarse por teléfono para "ponerse de acuerdo". Ya viste esta idea en OpenAPI (Semanas 1-2), en el archivo .proto de gRPC (Semana 3), en el esquema GraphQL (Semana 4), en el WSDL de SOAP (Semana 5) y en los eventos de dominio de una arquitectura orientada a eventos (Semana 6). Esta semana da un paso atrás y estudia el contrato como concepto en sí mismo: qué lo hace un buen contrato, cómo se decide su forma, qué lenguajes existen para describirlo formalmente, quién es responsable de custodiarlo cuando hay muchos productores y consumidores, y —el problema más delicado de todos— cómo cambiarlo sin romper a nadie que ya lo esté usando.

Este último punto, la compatibilidad hacia atrás (backward compatibility), es en la práctica profesional donde más incidentes de producción ocurren en sistemas de integración: un campo que se renombra, un tipo que cambia, un campo requerido que se agrega sin un valor por defecto — cualquiera de estos cambios, hecho sin disciplina, puede tumbar a decenas de consumidores que dependían del contrato anterior. La experiencia de laboratorio de esta semana modela exactamente este problema a pequeña escala: vas a definir el contrato de tu API de Productos como un JSON Schema explícito, validar solicitudes entrantes con una librería open source (ajv), y luego evolucionar ese esquema deliberadamente para sentir en carne propia la diferencia entre un cambio compatible y uno que rompe todo.

Conviene tener presente, antes de entrar en materia, que ninguna de las herramientas de esta semana (JSON Schema, Avro, Protobuf, un Schema Registry) resuelve el problema de la compatibilidad por sí sola: todas ellas son, en el fondo, mecanismos para hacer visible y verificable una disciplina que el equipo debe decidir seguir. Un equipo puede tener el Schema Registry mejor configurado del mundo y aun así romper a sus consumidores, si nadie revisa las alertas que ese Registry genera cuando rechaza un esquema incompatible. La tecnología acelera y automatiza la disciplina; no la reemplaza. Esta idea —que el valor real está en el proceso humano de revisión y coordinación, y la herramienta solo lo hace escalable— es, quizás, la lección más transferible de toda la semana a cualquier otro problema de ingeniería de software que enfrentes en tu carrera.

7.1. Contratos REST vs. gRPC

¿Qué es, exactamente, un "contrato" en integración de sistemas?

Un contrato de integración es una descripción formal, verificable y compartida de cómo dos sistemas deben comunicarse: qué operaciones existen, qué forma tienen los datos de entrada y salida, y qué errores pueden ocurrir. La palabra clave es formal: un contrato no es un documento de Word que un equipo le manda a otro por correo. Un contrato de integración moderno es un artefacto que una máquina puede leer y validar automáticamente — generar código a partir de él, validar mensajes contra él, o detectar si un cambio propuesto lo rompe.

Esta distinción importa porque cambia completamente el ciclo de vida del desarrollo. Si el contrato es solo un documento humano, la única forma de saber si dos sistemas dejaron de entenderse es que algo falle en producción. Si el contrato es un artefacto formal, ese mismo desacuerdo se puede detectar antes de desplegar — exactamente el mismo principio de "mover la validación lo más temprano posible" que ya viste con Swagger/OpenAPI en la Semana 1, ahora generalizado a cualquier tipo de contrato.

El contrato de REST: OpenAPI

Una API REST no tiene, por diseño, un contrato nativo obligatorio: HTTP no exige que describas tus endpoints en ningún formato particular. OpenAPI llenó ese vacío: un documento YAML/JSON que describe cada ruta, cada verbo HTTP, los parámetros esperados, y la forma del cuerpo de la petición/respuesta usando, precisamente, JSON Schema como sub-lenguaje.

```
paths:
  /productos/{id}:
    get:
      responses:
        '200':
          content:
            application/json:
              schema:
                type: object
                required: [id, nombre, precio]
                properties:
                  id: { type: integer }
                  nombre: { type: string }
                  precio: { type: number, minimum: 0 }
        '404':
          description: Producto no encontrado
```

En NestJS, este documento no se escribe a mano: se genera automáticamente a partir de los decoradores de @nestjs/swagger y de los tipos TypeScript de tus DTOs. Este es un enfoque code-first, concepto que se profundiza en la sección 7.2.

El contrato de gRPC: el archivo .proto

gRPC (Semana 3) invierte el orden: el contrato siempre se escribe primero, en un archivo .proto, y tanto el cliente como el servidor se generan a partir de él. No existe una forma de "usar gRPC

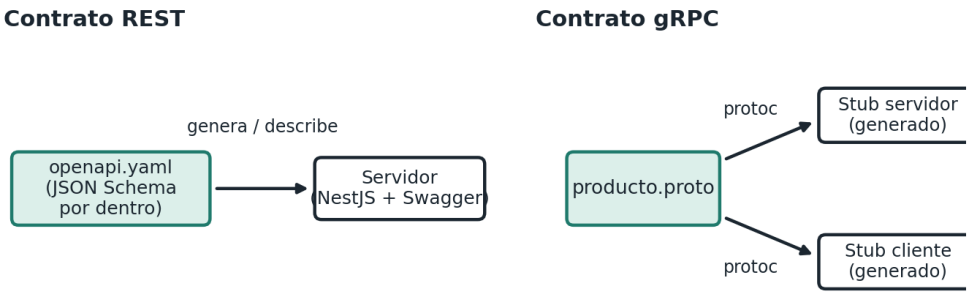
sin .proto" de la misma manera que sí existe una forma de "usar REST sin OpenAPI" — miles de APIs REST en producción no tienen ningún documento OpenAPI, y funcionan, aunque mal documentadas. gRPC fuerza la disciplina del contrato explícito desde el primer día; REST la permite, pero no la obliga.

```
syntax = "proto3";

message Producto {
  int32 id = 1;
  string nombre = 2;
  double precio = 3;
}

service ProductoService {
  rpc ObtenerProducto (ObtenerProductoRequest) returns (Producto);
}
```

Diagrama 1. Contrato REST (openapi.yaml genera/describe el servidor NestJS) vs. contrato gRPC (producto.proto genera, vía protoc, los stubs de servidor y cliente).



Contexto histórico: del contrato implícito al Contract-Driven Development

Durante buena parte de la década de los 2000, la integración entre sistemas dependía de contratos casi siempre implícitos: un equipo publicaba un servicio SOAP con su WSDL (Semana 5) y, a partir de ahí, cualquier cambio se comunicaba "a mano" — un correo, una reunión, un documento de Word actualizado manualmente. Cuando la industria migró masivamente a REST a inicios de la década de 2010, el péndulo se movió al extremo opuesto: muchas APIs REST se lanzaron sin ningún contrato formal, confiando en que la documentación humana (a menudo desactualizada) bastara. Los incidentes de integración que esto generó —clientes rotos por cambios no anunciados, equipos descubriendo incompatibilidades solo en producción— llevaron al surgimiento de lo que hoy se conoce como Contract-Driven Development (Desarrollo Dirigido por Contratos): la práctica de tratar el contrato como un artefacto de primera clase, versionado en el mismo repositorio que el código, validado automáticamente en cada cambio, y usado como fuente de verdad tanto para generar código como para generar pruebas.

OpenAPI (2015, evolución de Swagger) y la adopción masiva de gRPC/Protobuf en sistemas internos de empresas como Google y Netflix son, en gran medida, la respuesta de la industria a esa lección aprendida por la vía difícil. El propio auge de los Schema Registry (sección 7.4) en el ecosistema de streaming de eventos es una repetición de la misma historia, esta vez aplicada a los mensajes asíncronos en vez de a las llamadas síncronas.

Tipado fuerte vs. tipado débil: ¿cuál contrato es "más contrato"?

Entre OpenAPI y un .proto, ¿cuál es un contrato "más fuerte"? La respuesta depende de qué se garantiza y cuándo.

Tabla 1. OpenAPI vs. .proto como contrato

Aspecto	OpenAPI (REST)	.proto (gRPC)
¿Cuándo se verifica el tipo?	En tiempo de ejecución	En tiempo de compilación
¿El documento es obligatorio?	No — el servidor funciona sin él	Sí — sin él no hay stubs ni cliente

Si se desincronizan	Problema silencioso y frecuente	Prácticamente imposible (misma fuente)
Formato del payload	JSON (legible)	Binario (compacto, no legible)

La lección no es que gRPC sea "mejor": describir un contrato y hacerlo cumplir por la máquina son dos cosas distintas, y elegir la tecnología correcta depende de si la prioridad es la legibilidad humana y la compatibilidad universal con navegadores (REST) o la seguridad de tipos y el rendimiento en comunicación interna de alto volumen (gRPC).

Cuándo un contrato "más contrato" no es necesariamente mejor

Es tentador concluir que, dado que gRPC obliga a un contrato más estricto, siempre conviene usarlo. Esta rigidez tiene un costo: cambiar un .proto en producción exige regenerar y redistribuir clientes, mientras que un endpoint REST puede, en principio, empezar a devolver un campo adicional sin que ningún cliente existente se entere ni se rompa. Esta flexibilidad de REST no es un accidente: JSON es, por diseño, un formato abierto (se pueden agregar campos sin declarar un esquema nuevo), mientras que Protobuf es un formato cerrado por números de campo fijos (aunque, como verás en 7.5, también tiene sus propias reglas de evolución segura).

Catálogos de contratos en la industria

Así como en la Semana 1 vimos que el registro/descubrimiento de servicios (UDDI, y hoy los "developer portals") es una pieza estructural de SOA, las empresas que exponen APIs a gran escala mantienen catálogos públicos de sus contratos, precisamente para que cualquier integrador externo pueda generar clientes y validar compatibilidad sin tener acceso al código fuente:

Tabla 1b. Catálogos de contratos publicados por empresas reales

Empresa	Formato del catálogo	Qué resuelve
Stripe	OpenAPI + SDKs generados automáticamente en 8+ lenguajes	Cualquier integrador obtiene un cliente tipado sin escribir HTTP a mano
GitHub	OpenAPI público (api.github.com) + GraphQL SDL	Dos contratos distintos para dos estilos de consumo (REST y GraphQL) del mismo dominio
Twilio	OpenAPI por producto (SMS, Voz, Video)	Contratos independientes que evolucionan a ritmos distintos según el producto
Google Cloud	Protobuf/gRPC público para la mayoría de sus APIs internas y algunas externas	Contratos de alto rendimiento para integraciones entre servicios en la nube

El patrón común detrás de los cuatro casos es el mismo que estudiaste en 7.2: el contrato se trata como un producto en sí mismo, versionado y publicado deliberadamente, no como un efecto secundario accidental del código interno.

Ejercicio resuelto 7.1 — Elegir el contrato correcto

Enunciado: (a) una API pública que consumirán decenas de partners externos con distintos stacks; (b) comunicación interna de alto volumen entre Inventario y Precios, con picos de miles de solicitudes por segundo; (c) un panel de administración interno que un frontend React consume desde el navegador.

Solución: (a) OpenAPI/REST — audiencia heterogénea, el navegador y herramientas HTTP genéricas deben inspeccionar el contrato fácilmente. (b) gRPC/Protobuf — comunicación interna de alto volumen entre servicios que controlas tú mismo. (c) OpenAPI/REST — un navegador no puede invocar gRPC nativamente sin una capa de traducción adicional.

Taller en grupo 7.1 — Auditar un contrato ajeno con IA

En parejas, tomen el .proto de la Semana 3 y pidan a un agente de IA: "Actúa como un integrador que solo puede ver este .proto, sin acceso al código del servidor. Dime qué operaciones puedes inferir, qué preguntas le harías al equipo dueño del servicio, y qué información NO puedes deducir solo del contrato." Verificación obligatoria: contrasten la respuesta contra el código real del servidor e identifiquen al menos una cosa que el .proto no comunica.

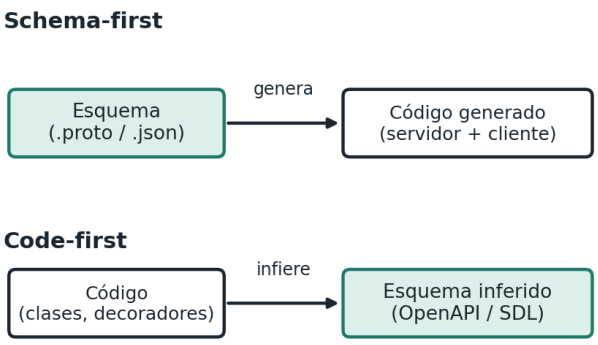
7.2. Enfoques Schema-first y Code-first

Ya usaste ambos enfoques sin nombrarlos explícitamente: en las Semanas 1-2, el OpenAPI de tu API NestJS se genera a partir de tus decoradores y DTOs — primero el código, el esquema aparece como consecuencia (code-first). En la Semana 3, escribiste el .proto antes de que existiera una línea de implementación (schema-first). En la Semana 4, GraphQL con NestJS/Apollo ofrece ambos caminos.

Schema-first (contrato primero): el equipo diseña y acuerda el contrato antes de escribir ninguna implementación, frecuentemente en un repositorio propio, independiente del código de cualquiera de los servicios que lo implementan.

Code-first (código primero): el contrato se deriva automáticamente del código real. El desarrollador nunca edita el documento del contrato directamente: lo edita indirectamente, cambiando el código.

Diagrama 2. Schema-first (el esquema genera el código) vs. Code-first (el código infiere el esquema).



Ventajas y riesgos de cada enfoque

Schema-first: ventajas. Permite que equipos distintos trabajen en paralelo desde el primer día: en cuanto el contrato está acordado, el equipo consumidor puede generar un cliente contra un mock mientras el proveedor implementa la lógica real. Es especialmente valioso en organizaciones grandes o distribuidas geográficamente. El contrato queda como un artefacto de diseño deliberado, revisable en una reunión de arquitectura antes de invertir esfuerzo de programación.

Schema-first: riesgos. El contrato puede desincronizarse de la implementación real sin disciplina. Mantener el archivo de esquema es trabajo adicional explícito.

Code-first: ventajas. El contrato nunca puede desincronizarse, porque se genera desde la implementación. Es más rápido para empezar y encaja con el ciclo iterativo de equipos pequeños.

Code-first: riesgos. El diseño del contrato queda subordinado al diseño interno del código — es fácil terminar con un contrato "feo" que refleja la estructura de las clases del servidor. Dificulta el trabajo verdaderamente paralelo entre equipos.

Tabla 2. Schema-first vs. Code-first

Criterio	Schema-first	Code-first
¿Quién manda?	El esquema	El código
Trabajo paralelo entre equipos	Excelente	Limitado
Riesgo de desincronización	Alto sin disciplina	Prácticamente nulo
Calidad del diseño del contrato	Potencialmente mejor	Hereda defectos del código
Ejemplos del curso	.proto (S3), GraphQL SDL a mano	Swagger (S1-2), GraphQL code-first (S4)

Estudio de caso: cómo Stripe diseña su contrato schema-first

Stripe, la pasarela de pagos usada por millones de comercios en el mundo, es frecuentemente citada como uno de los mejores ejemplos de diseño de contrato schema-first a gran escala. Cada versión de su API se identifica con una fecha (por ejemplo, 2024-06-20), no con un simple número de versión: cada cambio incompatible que el equipo interno de Stripe hace a su modelo de datos se agrupa en una nueva "versión con fecha", y cada cuenta de comercio queda fijada a la versión con la que se integró originalmente, hasta que decide migrar deliberadamente. Stripe mantiene, internamente, una capa de transformación que traduce entre todas las versiones históricas y el modelo de datos interno más reciente — en la práctica, un caso real y masivo del patrón Message Translator que ya viste en la Semana 5 (ESB), aplicado esta vez a la evolución de un contrato REST en el tiempo, en vez de a la traducción entre dos protocolos distintos.

La lección de este caso para un proyecto mucho más pequeño como el tuyo es la misma en esencia: decidir explícitamente cómo vas a versionar tus cambios incompatibles (por fecha, por número de versión en la URL, por cabecera HTTP) es una decisión de arquitectura que conviene tomar temprano, no algo que se improvisa la primera vez que un cambio rompe algo en producción.

GitHub, por su parte, ilustra una cuarta estrategia de versionamiento: negociación de contenido mediante cabeceras HTTP personalizadas (por ejemplo, `Accept: application/vnd.github+json; version=2022-11-28`). En vez de cambiar la URL o esperar a un número de versión mayor, el cliente declara explícitamente en cada petición qué versión del contrato espera recibir, y el servidor decide cómo responder según esa cabecera. Esta estrategia tiene una ventaja sutil sobre versionar en la URL: la misma ruta (`GET /repos/{owner}/{repo}`) puede servir simultáneamente a clientes de distintas versiones sin necesidad de mantener rutas duplicadas (`/v1/repos/...`, `/v2/repos/...`) — el costo es que la versión queda "escondida" en una cabecera en vez de ser visible a simple vista en la URL, lo cual dificulta un poco la depuración manual con herramientas simples como el navegador.

¿Cuál usar en la práctica de hoy?

La experiencia de laboratorio usa deliberadamente schema-first: vas a escribir el JSON Schema de la entidad Producto en un archivo .json separado, antes de tocar el código del controlador. Esto es intencional: quiere que sientas la experiencia de "el contrato existe como un artefacto independiente que yo mismo debo mantener actualizado" — algo que la comodidad de los decoradores automáticos de NestJS tiende a ocultar.

Ejercicio resuelto 7.2 — Detectar cuál enfoque se usó

Enunciado: El Proyecto A tiene un openapi.yaml con un README que dice "no edites este archivo, es generado en el build". El Proyecto B tiene un api-contract.yaml que un equipo de arquitectura mantiene manualmente, y un CI que falla si la implementación no lo cumple.

Solución: A es code-first (la instrucción de "no editar, es generado" es la señal inequívoca). B es schema-first (el contrato vive como artefacto propio y el CI verifica que la implementación se ajuste a él).

Taller en grupo 7.2 — Diseñar un contrato schema-first con IA

En equipos de 3, elijan una entidad propia y, antes de escribir código, redacten a mano un borrador de su contrato. Pidan a la IA que actúe como el equipo consumidor y señale ambigüedades. Verificación obligatoria: identifiquen si ya habían anticipado esas ambigüedades al diseñar el contrato.

7.3. JSON Schema, Avro y Protobuf

JSON Schema: describiendo la forma de un documento JSON

JSON Schema es un vocabulario, escrito en el propio formato JSON, para describir la estructura, los tipos y las restricciones de otro documento JSON. Es el lenguaje que usa OpenAPI "por dentro" para describir los cuerpos de petición/respuesta, y es exactamente lo que vas a escribir a mano en la práctica de hoy.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Producto",
  "type": "object",
  "required": ["nombre", "precio"],
  "properties": {
    "id": { "type": "integer", "minimum": 1 },
    "nombre": { "type": "string", "minLength": 1, "maxLength": 120 },
    "precio": { "type": "number", "minimum": 0 },
    "categoria": { "type": "string", "enum": ["electronica", "oficina", "hogar"] }
  },
  "additionalProperties": false
}
```

Tabla 3. Vocabulario esencial de JSON Schema

Palabra clave	Qué controla
type	El tipo de dato esperado (object, string, number, integer, boolean, array, null).
required	Lista de propiedades que deben estar presentes.
properties	El esquema de cada campo individual.
minimum / maximum	Límites numéricos inclusivos.
minLength / maxLength	Límites de longitud para cadenas de texto.
enum	Restringe el valor a un conjunto fijo de opciones.
pattern	Expresión regular que el texto debe cumplir.
additionalProperties	Si es false, rechaza campos no declarados — detecta errores de tipeo.
$ref	Referencia a otro esquema, para reutilizar definiciones.
format	Formatos semánticos (email, date-time, uri), validados con ajv-formats.

JSON Schema es, deliberadamente, JSON describiendo JSON: no requiere una sintaxis nueva, solo un vocabulario de palabras clave reservadas, fácil de generar programáticamente, de versionar en Git, y de validar con librerías open source disponibles en prácticamente cualquier lenguaje (en Node.js, la más usada es ajv, que usarás en la práctica de hoy).

JSON Schema: esquemas anidados y palabras clave de composición

Los objetos del mundo real casi nunca son planos. Un Pedido, por ejemplo, contiene una lista de líneas, y cada línea referencia un Producto. JSON Schema modela esto con arreglos (type: array + items) y con $ref para reutilizar la definición de Producto sin duplicarla:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Pedido",
  "type": "object",
  "required": ["clienteId", "items"],
  "properties": {
    "clienteId": { "type": "integer" },
    "items": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "producto.schema.json" }
    }
  }
}
```

Además de la composición estructural, JSON Schema ofrece palabras clave para combinar validaciones alternativas o simultáneas, útiles cuando un campo puede tomar más de una "forma":

Tabla 3b. Palabras clave de composición en JSON Schema

Palabra clave	Significado
allOf	El dato debe cumplir TODOS los sub-esquemas listados (equivalente a herencia múltiple de restricciones).
oneOf	El dato debe cumplir EXACTAMENTE UNO de los sub-esquemas listados.
anyOf	El dato debe cumplir AL MENOS UNO de los sub-esquemas listados.
not	El dato NO debe cumplir el sub-esquema indicado.

Estas palabras clave permiten modelar, por ejemplo, un campo metodoPago que puede ser "tarjeta" (con número y fecha de expiración) o "transferencia" (con número de cuenta), usando oneOf para expresar que el dato debe encajar en una de las dos formas, pero no en ambas ni en ninguna.

Apache Avro: el esquema binario del mundo Kafka

Avro es un sistema de serialización de datos, también basado en un esquema JSON, cuyo propósito es distinto al de JSON Schema: mientras JSON Schema valida un documento JSON existente, Avro serializa los datos a un formato binario compacto, usando el esquema para saber cómo interpretar esos bytes.

```
{
  "type": "record",
  "name": "Producto",
  "fields": [
    { "name": "id", "type": "int" },
    { "name": "nombre", "type": "string" },
    { "name": "precio", "type": "double" },
    { "name": "categoria", "type": ["null", "string"], "default": null }
  ]
}
```

La diferencia fundamental frente a Protobuf: el esquema viaja (o se referencia) junto con el dato. En Protobuf, el consumidor necesita tener de antemano el mismo .proto que usó el productor, porque los bytes por sí solos no llevan pista de qué campo es cuál (solo un número de campo). En Avro, el esquema completo está disponible junto al dato o registrado en un Schema Registry (sección 7.4) al que productor y consumidor consultan por su identificador. Esto hace que Avro sea especialmente popular junto a Kafka (Semana 6): cada mensaje puede resolverse contra el esquema exacto con el que fue escrito, gracias a las reglas de resolución de esquemas que Avro define explícitamente.

Protobuf: repaso desde la óptica de esta semana

Ya conoces Protobuf desde la Semana 3 como el lenguaje de contrato de gRPC. Lo que agrega esta semana es la óptica de evolución del esquema: cada campo de un mensaje Protobuf tiene un número asociado (int32 id = 1;), y ese número —no el nombre del campo— es lo que realmente viaja en el binario. Mientras el número de campo no cambie ni se reutilice, se pueden renombrar campos, agregar campos nuevos o incluso quitar campos (marcándolos como reserved) sin romper la compatibilidad binaria con clientes que usan una versión anterior del .proto.

Diagrama 3. JSON Schema (valida JSON de texto) vs. Avro (serializa a binario, esquema vía Registry, Kafka) vs. Protobuf (serializa a binario, esquema en el cliente, gRPC).

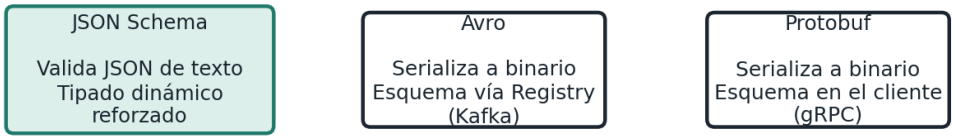


Tabla 4. JSON Schema vs. Avro vs. Protobuf

Criterio	JSON Schema	Avro	Protobuf
Propósito	Validar JSON existente	Serializar a binario	Serializar + generar stubs RPC
Formato en tránsito	Texto (JSON)	Binario	Binario
¿Esquema viaja con el dato?	Dato ya autodescriptivo	Vía Schema Registry	No — el cliente ya lo tiene
Tipado	Dinámico reforzado	Fuerte	Fuerte, en compilación
Ecosistema típico	APIs REST (OpenAPI)	Streaming (Kafka)	RPC de alto rendimiento (gRPC)
Uso en este curso	Práctica de hoy (ajv)	Referencia (no implementado)	Semana 3 (gRPC)

Ejercicio resuelto 7.3 — Elegir el lenguaje de esquema correcto

Enunciado: (a) validar el body de un POST /productos antes de procesarlo; (b) publicar millones de eventos de clic por segundo a Kafka minimizando el tamaño del mensaje; (c) definir el contrato binario de un nuevo servicio gRPC interno de alto rendimiento.

Solución: (a) JSON Schema — el body ya llega como JSON, solo hay que validarlo. (b) Avro — el volumen masivo hace importar el tamaño compacto, y Kafka tiene soporte de primera clase con Schema Registry. (c) Protobuf — es el lenguaje de contrato nativo de gRPC.

Ejercicio resuelto 7.3b — Leyendo un esquema Avro real de principio a fin

Igual que en la Semana 5 practicaste leer un WSDL ajeno sin ver el código, aquí conviene practicar la habilidad equivalente con Avro.

Enunciado: te entregan este esquema Avro de un sistema de eventos de terceros. Sin ver el código productor, responde: (a) ¿qué campos son opcionales?, (b) ¿qué pasaría si un consumidor viejo (sin el campo canalOrigen) lee un evento nuevo?

```
{
  "type": "record",
  "name": "PedidoCreado",
  "fields": [
    { "name": "pedidoId", "type": "string" },
    { "name": "total", "type": "double" },
    { "name": "canalOrigen", "type": ["null", "string"], "default": null }
  ]
}
```

Solución razonada: (a) canalOrigen es el único campo opcional: su tipo es una unión ["null", "string"] con default: null, la forma estándar en Avro de declarar "este campo puede no venir". pedidoId y total son requeridos (tipos simples sin unión con null). (b) Un consumidor viejo que no conoce canalOrigen simplemente lo ignora al leer el evento nuevo — esto es exactamente forward compatibility en acción (sección 7.5): el esquema nuevo agregó un campo opcional, y el consumidor viejo sigue funcionando sin cambios.

Taller en grupo 7.3 — Traducir un contrato entre lenguajes de esquema con IA

En parejas, tomen el JSON Schema de Producto y pidan a la IA que lo traduzca a Avro y a Protobuf equivalentes. Verificación obligatoria: revisen manualmente qué restricciones (enum, minimum/maximum) se "perdieron" o aproximaron en la traducción, y por qué.

7.4. Schema Registry y versionamiento

El problema que resuelve un Schema Registry

En una arquitectura orientada a eventos (Semana 6) con decenas de productores y consumidores publicando y leyendo del mismo Event Bus, surge una pregunta operativa concreta: ¿dónde vive la versión "oficial" de cada esquema, y cómo se asegura que un productor no publique un evento con un esquema incompatible con lo que los consumidores ya esperan? Un Schema Registry (el más conocido es el de Confluent, para Kafka) es un servicio centralizado que:

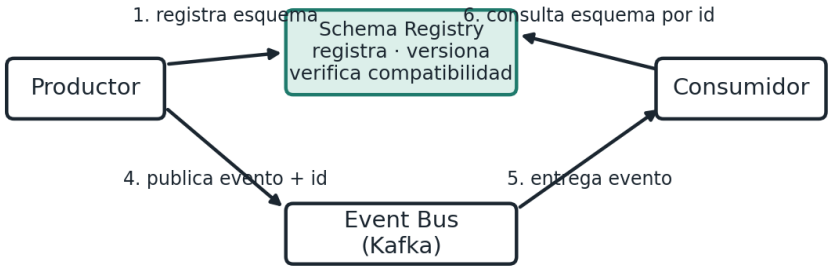
1. Almacena cada versión de cada esquema que los productores registran antes de publicar.

2. Asigna un identificador único a cada versión de esquema.

3. Verifica automáticamente si una nueva versión es compatible con las anteriores, según una política configurada (backward, forward o full — sección 7.5), y rechaza el registro si no lo es.

4. Permite que los consumidores resuelvan, a partir del identificador embebido en cada mensaje, exactamente qué esquema usar para deserializarlo.

Diagrama 4. Flujo con Schema Registry: el productor registra el esquema (1) y publica el evento con su id (4); el consumidor recibe el evento (5) y consulta el esquema por id (6) antes de deserializar.



Si la verificación de compatibilidad falla, el registro se rechaza en el momento del registro, no cuando algún consumidor desconocido intente leer el mensaje meses después — esta es la razón de ser del Schema Registry: mover el error de "producción, silencioso, tarde" a "desarrollo/despliegue, explícito, temprano".

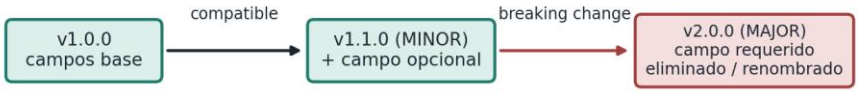
Versionamiento de esquemas: aplicando SemVer a los contratos

Tabla 5. Versionamiento semántico aplicado a esquemas

Cambio	Tipo de versión	Ejemplo
Agregar un campo opcional nuevo	MINOR (compatible)	Agregar categoria opcional a Producto
Agregar un valor a un enum no exhaustivo	MINOR (compatible)	Agregar "jardin" a categorías
Corregir descripción o metadatos	PATCH	Corregir un typo
Eliminar campo requerido / cambiar tipo	MAJOR (breaking)	Quitar precio, o cambiarlo a string

Esta clasificación en tres niveles no es solo cosmética: MAJOR es exactamente la señal que un pipeline de CI (sección 7.5) debe usar para bloquear un despliegue hasta que todos los consumidores conocidos hayan migrado, mientras que MINOR y PATCH pueden desplegarse sin coordinación adicional.

Diagrama 5. Línea de tiempo de versionamiento: v1.0.0 evoluciona de forma compatible a v1.1.0 (MINOR), y de forma incompatible a v2.0.0 (MAJOR).



Versionamiento en el contrato REST: repaso desde la Semana 2

Vale la pena conectar esta sección con el versionamiento de APIs REST que ya viste en la Semana 2 (/v1/productos vs. /v2/productos): es la misma disciplina de "cambio incompatible = nueva versión explícita", aplicada al nivel de toda la API en vez de a un esquema individual. La diferencia es de granularidad: un Schema Registry versiona esquemas individuales; el versionamiento de rutas REST versiona la API completa — en sistemas grandes, ambos mecanismos conviven.

Las políticas de compatibilidad reales de un Schema Registry

El Schema Registry de Confluent (la implementación de referencia del ecosistema Kafka) no ofrece solo tres políticas simples, sino siete niveles configurables por tópico, que vale la pena conocer porque cualquier otro Schema Registry serio (incluyendo implementaciones propias) termina reinventando una variante de esta misma lista:

Tabla 5b. Niveles de compatibilidad de un Schema Registry (modelo Confluent)

Nivel	Garantía
NONE	Sin verificación — cualquier esquema se acepta, incluyendo cambios que rompan todo.
BACKWARD	Los consumidores con el esquema más reciente pueden leer datos escritos con la versión inmediatamente anterior.
BACKWARD_TRANSITIVE	Igual que BACKWARD, pero contra TODAS las versiones anteriores, no solo la inmediata.
FORWARD	Los consumidores con la versión anterior pueden leer datos escritos con el esquema más reciente.
FORWARD_TRANSITIVE	Igual que FORWARD, pero contra todas las versiones futuras previstas.
FULL	Se cumplen BACKWARD y FORWARD simultáneamente, contra la versión inmediata.
FULL_TRANSITIVE	Se cumplen ambas garantías contra el

	historial completo de versiones.

La política "transitiva" importa especialmente en sistemas con muchos consumidores que se actualizan en momentos distintos: si un consumidor lleva meses sin actualizarse (una situación extremadamente común en organizaciones grandes), BACKWARD_TRANSITIVE es la única garantía que asegura que ese consumidor rezagado seguirá funcionando después del próximo cambio de esquema, no solo después del cambio inmediatamente anterior.

Ejercicio resuelto 7.4 — Clasificar cambios de esquema

Enunciado: (a) agregar descripcionLarga opcional; (b) cambiar precio de number a integer; (c) agregar "jardin" al enum de categoria; (d) hacer categoria requerido.

Solución: (a) MINOR — no rompe a nadie. (b) MAJOR — estrechar el tipo puede rechazar datos antes válidos (19.99). (c) MINOR con advertencia — compatible solo si los consumidores toleran valores desconocidos. (d) MAJOR — cualquier petición anterior sin ese campo sería rechazada.

Taller en grupo 7.4 — Diseñar la política de versionamiento de un dominio propio con IA

En equipos de 3-4, listen 5 cambios de esquema hipotéticos de un dominio propio y clasifíquenlos MAJOR/MINOR/PATCH antes de usar IA. Pidan a la IA que actúe como el administrador de un Schema Registry en modo "backward" y decida si aprueba o rechaza cada uno. Verificación obligatoria: resuelvan en equipo cualquier discrepancia con su propia clasificación.

7.5. Validación automática y backward compatibility

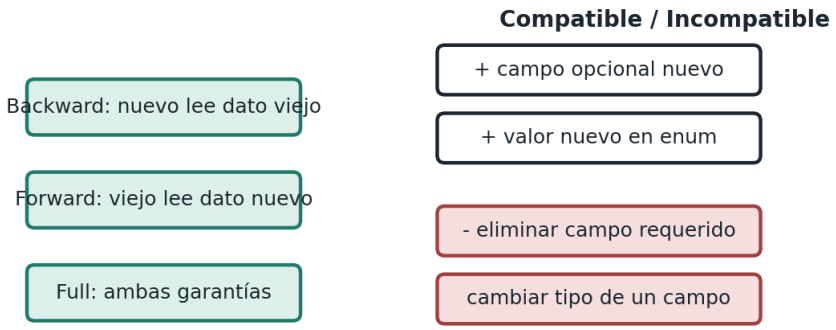
Los tres tipos de compatibilidad

Backward compatibility: un consumidor con el esquema nuevo puede leer correctamente datos escritos con el esquema viejo. Es la garantía que practicarás hoy.

Forward compatibility: un consumidor que todavía usa el esquema viejo puede leer correctamente datos escritos con el esquema nuevo — el consumidor viejo ignora los campos nuevos sin fallar.

Full compatibility: ambas garantías se cumplen simultáneamente. Es la política más estricta y la más segura cuando no se puede coordinar el orden de despliegue.

Diagrama 6. Reglas de compatibilidad: las tres políticas (izquierda) y ejemplos de cambios compatibles vs. incompatibles (derecha).



Reglas concretas: qué rompe compatibilidad y qué no

Cambios típicamente seguros (backward-compatible):

- Agregar un campo opcional nuevo (con valor por defecto si el formato lo requiere).
- Agregar un valor nuevo a un enum, si los consumidores toleran valores desconocidos.
- Relajar una restricción (aumentar un maxLength, quitar un pattern demasiado estricto).
- Agregar una operación/endpoint nuevo que no interfiere con los existentes.

Cambios típicamente incompatibles (breaking changes):

- Eliminar un campo que algún consumidor todavía lee.
- Cambiar el tipo de un campo existente.
- Renombrar un campo (indistinguible, para el esquema, de eliminar uno y agregar otro).
- Hacer requerido un campo que antes era opcional.
- Estrechar una restricción de forma que datos previamente válidos dejen de serlo.
- En Protobuf: reutilizar el número de un campo eliminado para uno con significado distinto.

Validación automática de requests entrantes

La validación automática es el mecanismo que hace cumplir el contrato en cada petición, en tiempo real. En NestJS/Node, la práctica de hoy usa ajv (Another JSON Validator), la librería de validación de JSON Schema más usada del ecosistema JavaScript.

```
const Ajv = require('ajv');
const addFormats = require('ajv-formats');
const ajv = new Ajv({ allErrors: true });
addFormats(ajv);

const schema = require('./producto.schema.json');
const validate = ajv.compile(schema);

const valido = validate({ nombre: 'Teclado mecánico', precio: 45.90 });
```

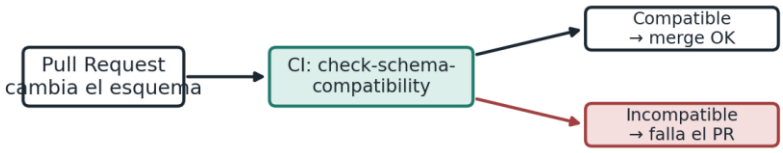
```
if (!valido) {
  console.log(validate.errors);
}
```

Es importante contrastar este enfoque con el de la Semana 2: allí, la validación se hacía con DTOs decorados con class-validator — las reglas viven dentro del código TypeScript, como anotaciones sobre una clase. Con ajv y JSON Schema, las reglas viven en un archivo .json separado, independiente del lenguaje del servidor. Esta es la diferencia práctica entre "contrato como código" (DTOs) y "contrato como dato" (JSON Schema) — ambos legítimos, y elegir uno u otro es, de nuevo, una decisión de diseño.

Validación de compatibilidad en el pipeline de CI

La práctica más avanzada consiste en agregar un paso al pipeline de CI que, antes de fusionar un cambio, compare el esquema nuevo contra el esquema actualmente en producción y falle explícitamente si detecta una regla de incompatibilidad violada. Esto traslada la pregunta "¿este cambio rompe a alguien?" de una discusión informal en revisión de código a una verificación automática y objetiva.

Diagrama 7. Pipeline de CI: un Pull Request que cambia el esquema pasa por check-schema-compatibility, que aprueba el merge o hace fallar el PR.



En la práctica de hoy construirás una versión simplificada de este chequeo: un pequeño script Node que compara dos versiones del JSON Schema de Producto y aplica, a mano, las reglas heurísticas descritas arriba. No reemplaza una herramienta de producción real, pero da el modelo mental exacto que esas herramientas automatizan a mayor escala. Un esqueleto de esa heurística, en JavaScript plano, se ve así:

```
function esCompatibleBackward(schemaViejo, schemaNuevo) {
  const problemas = [];
  const reqViejo = new Set(schemaViejo.required || []);
  const reqNuevo = new Set(schemaNuevo.required || []);

  // Requerido nuevo que no existia antes => breaking change
  for (const campo of reqNuevo) {
    if (!reqViejo.has(campo) && !(campo in (schemaViejo.properties || {}))) {
      problemas.push(`Campo requerido nuevo sin default: ${campo}`);
    }
  }
  // Campo que existia y desaparecio => breaking change
  for (const campo of Object.keys(schemaViejo.properties || {})) {
    if (!(campo in (schemaNuevo.properties || {}))) {
      problemas.push(`Campo eliminado: ${campo}`);
    }
  }
  // Cambio de tipo en un campo existente => breaking change
  for (const campo of Object.keys(schemaViejo.properties || {})) {
    const tipoViejo = schemaViejo.properties[campo]?.type;
    const tipoNuevo = schemaNuevo.properties?.[campo]?.type;
    if (tipoNuevo && tipoViejo !== tipoNuevo) {
      problemas.push(`Tipo cambiado en ${campo}: ${tipoViejo} -> ${tipoNuevo}`);
    }
  }
  return { compatible: problemas.length === 0, problemas };
}
```

Esta función deliberadamente no cubre todos los casos posibles (por ejemplo, no analiza cambios dentro de un $ref anidado, ni restricciones numéricas más estrictas) — es un punto de partida pedagógico, no una librería de producción. Herramientas reales como el propio verificador del Confluent Schema Registry, o buf breaking para Protobuf, implementan reglas

mucho más exhaustivas, pero el principio que aplican es exactamente el mismo que acabas de leer: comparar estructuralmente dos versiones de un contrato y señalar cualquier diferencia que viole la política de compatibilidad configurada.

Ejercicio resuelto 7.5 — Diagnosticar un incidente de compatibilidad

Enunciado: Un equipo despliega la v2 de su esquema Pedido, agregando metodoEnvio como requerido, sin coordinarse con tres consumidores existentes. A las pocas horas, dos de los tres fallan al procesar eventos nuevos.

Solución: Se violó backward compatibility: los consumidores escritos contra v1 reciben mensajes que exigen un campo que antes no existía. El cambio seguro equivalente habría sido declarar metodoEnvio como opcional, con un valor por defecto razonable aplicado por los consumidores cuando el campo no está presente — MINOR, no MAJOR.

Taller en grupo 7.5 — Simular un Schema Registry con IA

En parejas, definan v1 y v2 de un esquema propio, con al menos un cambio compatible y uno incompatible mezclados en v2. Pidan a la IA que actúe como el motor de verificación en modo "backward" y liste cada cambio con su clasificación. Verificación obligatoria: implementen a mano al menos una de las comparaciones y confirmen si coincide con la IA — exactamente la habilidad del Paso 6 (reto) de la práctica.

De los contratos a las pruebas: un adelanto de la Semana 9

Todo lo que has visto hoy —contratos formales, esquemas versionados, verificación de compatibilidad— responde a la pregunta "¿el mensaje tiene la forma correcta?". Queda pendiente una pregunta relacionada pero distinta, que retomarás formalmente en la Semana 9 (Pruebas de integración y contract testing): "¿el proveedor y el consumidor realmente se comportan como el contrato promete?". Un JSON Schema puede confirmar que un campo precio es un number, pero no puede confirmar, por ejemplo, que ObtenerProductos siempre devuelva al menos un elemento cuando el catálogo no está vacío, o que un código de error específico se use consistentemente. Esa verificación de comportamiento —más allá de la forma— es exactamente lo que herramientas como Pact (contract testing entre consumidor y proveedor) añaden encima de la disciplina de esquemas que construiste esta semana. Dicho de otro modo: el esquema es la gramática del contrato; el contract testing verifica que ambas partes efectivamente hablan ese idioma de la misma manera.

Errores comunes al diseñar y evolucionar contratos

Antes de pasar al resumen, vale la pena reunir en un solo lugar los errores que con más frecuencia se observan en proyectos reales (y que la rúbrica de la práctica de hoy penaliza explícitamente):

- Cambiar el contrato y el código en el mismo commit sin que nadie revise el impacto en los consumidores existentes — el equivalente a publicar sin pasar por el paso 2 del Diagrama 7.
- Usar additionalProperties: false en un esquema que se sabe va a evolucionar con frecuencia, sin un plan claro de cómo se agregarán campos nuevos más adelante.

- Confundir "documentar el contrato" con "hacerlo cumplir": tener un openapi.yaml bonito no sirve de nada si el servidor real no se valida contra él en cada build.
- Renombrar un campo pensando que es un cambio "cosmético" — para cualquier motor de validación, renombrar es indistinguible de eliminar uno y agregar otro (ver 7.5).
- No versionar el propio archivo de esquema en el control de versiones (Git), tratándolo como un artefacto generado y descartable en vez de como código fuente de primera clase.
- Diseñar el contrato pensando únicamente en el primer consumidor conocido, sin dejar espacio de extensión (campos opcionales, enums no exhaustivos) para consumidores futuros que aún no existen.

Caso de estudio integrador: modernizando un monolito de facturación con contratos explícitos

Cerramos la semana con un caso que integra las cinco secciones anteriores. Una empresa mediana tiene un monolito de facturación de 8 años de antigüedad, sin ningún contrato formal: los pocos sistemas que lo consumen (un portal web y un job nocturno de contabilidad) simplemente "saben" qué forma tiene la respuesta, porque alguien leyó el código fuente alguna vez. El equipo decide modernizar la integración sin reescribir el monolito, siguiendo cuatro pasos que resumen exactamente el contenido de esta semana:

1. Documentar el contrato existente en OpenAPI (schema-first, aplicado retroactivamente): un ingeniero inspecciona las respuestas reales del sistema y escribe el JSON Schema de cada entidad (Factura, Cliente, LineaFactura), incluyendo restricciones que el código nunca declaró explícitamente pero que siempre cumplió en la práctica (por ejemplo, que el total nunca es negativo).

2. Agregar validación automática (sección 7.5) delante del monolito, usando un pequeño proxy que valida cada respuesta saliente contra el esquema recién documentado antes de reenviarla a los consumidores — esto expone, de inmediato, tres inconsistencias que el equipo desconocía (un campo que a veces venía como null y a veces como cadena vacía).

3. Definir la política de compatibilidad (BACKWARD, sección 7.4) para cualquier cambio futuro al contrato documentado, y agregar un chequeo de compatibilidad al pipeline de CI del propio proxy, de forma que ningún cambio futuro pueda romper al portal web o al job de contabilidad sin que alguien lo note explícitamente antes de desplegar.

4. Publicar el contrato como un artefacto versionado en un repositorio propio, independiente del código del monolito, de forma que cualquier consumidor nuevo (por ejemplo, una futura app móvil) pueda generar su cliente contra el contrato documentado sin tener que leer el código fuente del monolito ni coordinarse manualmente con su equipo.

Nótese que en ningún momento de este proceso se reescribió una sola línea de lógica de negocio del monolito: el valor completo de esta modernización proviene únicamente de hacer explícito y verificable un contrato que, hasta ese momento, solo existía de forma implícita en la cabeza de quienes habían leído el código fuente. Este es, en esencia, el mismo argumento que

ya viste en la Semana 5 sobre por qué introducir un ESB delante de un núcleo bancario legado es más pragmático que reescribirlo — aquí se aplica el principio análogo al nivel del contrato de datos en vez de al nivel de la infraestructura de mensajería.

Resumen de la semana

Tabla 6. Resumen por sección

Sección	Idea central
7.1	Un contrato explícito, verificable por máquina, permite que sistemas distintos se entiendan sin coordinación manual.
7.2	Schema-first y code-first son dos formas legítimas de llegar al mismo contrato, con trade-offs opuestos.
7.3	JSON Schema valida JSON textual; Avro y Protobuf serializan a binario compacto, con distintas estrategias de resolución de esquema.
7.4	Un Schema Registry centraliza el versionamiento y rechaza esquemas incompatibles antes de llegar a producción.
7.5	Backward, forward y full compatibility son garantías distintas; validar automáticamente (ajv, o CI) es lo que las hace cumplir.

Preguntas frecuentes

¿JSON Schema reemplaza a class-validator? No necesariamente — son dos formas de resolver el mismo problema. class-validator integra la validación en las clases TypeScript; JSON Schema la externaliza a un artefacto independiente del lenguaje.

¿Necesito un Schema Registry real para un proyecto pequeño? No. Resuelve un problema de coordinación entre muchos equipos y consumidores — para un proyecto de un solo equipo, la disciplina manual es suficiente.

¿Avro solo sirve para Kafka? No, pero es donde se usa casi exclusivamente en la práctica: su diseño está pensado para streaming de eventos de alto volumen.

¿Qué pasa si dos equipos no se ponen de acuerdo sobre quién es "dueño" del contrato? Es un problema de gobernanza, no técnico, y suele resolverse aplicando el mismo principio que en la Semana 1 vimos para el diseño de servicios: el equipo dueño del dominio de negocio al que pertenece la entidad (por ejemplo, el equipo de Catálogo para Producto) es quien controla el contrato; los equipos consumidores participan en su diseño, pero no lo controlan unilateralmente.

¿Vale la pena usar JSON Schema en un proyecto que ya usa TypeScript con tipos estrictos? Depende de dónde necesites que viva el contrato: los tipos de TypeScript solo existen en tiempo de compilación y desaparecen en el JavaScript final, así que no pueden validar datos que lleguen en tiempo de ejecución (por ejemplo, desde una petición HTTP externa). JSON Schema (vía ajv)

sí valida en tiempo de ejecución — por eso ambos mecanismos son complementarios, no competidores, en un mismo proyecto TypeScript.

Glosario de términos

- Contrato: descripción formal y verificable por máquina de cómo dos sistemas deben comunicarse.
- Esquema: representación formal, en un lenguaje específico, de la forma que deben tener los datos.
- Schema-first: el contrato se diseña antes de escribir la implementación.
- Code-first: el contrato se genera a partir del código ya escrito.
- Backward compatibility: un consumidor nuevo puede leer datos escritos con el esquema viejo.
- Forward compatibility: un consumidor viejo puede leer datos escritos con el esquema nuevo.
- Breaking change: modificación al esquema que rompe alguna garantía de compatibilidad.
- Schema Registry: servicio que almacena, versiona y verifica la compatibilidad de esquemas.
- ajv: librería open source de validación de JSON Schema para JavaScript/TypeScript.
- Avro: sistema de serialización binaria basado en esquemas JSON, usado junto a Kafka.

Referencias

- JSON Schema Organization. (2024). JSON Schema Specification. https://json-schema.org
- Confluent. (2024). Schema Registry Documentation.
- Apache Software Foundation. (2024). Apache Avro Specification. https://avro.apache.org
- Google. (2024). Protocol Buffers — Updating a Message Type.
- ajv (npm). https://ajv.js.org
- Fowler, M. (2023). Tolerant Reader Pattern.
- Stripe. (2024). API Versioning Documentation.

Autoevaluación

1. Explica con tus palabras por qué un .proto es "prescriptivo" y un openapi.yaml generado automáticamente es "descriptivo" — ¿qué implicación práctica tiene esa diferencia si el código y el documento se desincronizan?

2. Da un ejemplo propio (no del capítulo) de un cambio de esquema que sea backward-compatible pero no forward-compatible.

3. ¿Por qué Avro necesita un Schema Registry (o un esquema embebido) para que el consumidor pueda leer el dato, mientras que JSON Schema no impone ese requisito sobre el JSON que valida?

4. Describe, sin ver la sección 7.5, las tres políticas de compatibilidad principales (backward, forward, full) y en qué escenario de despliegue conviene cada una.

5. En tu propio proyecto (o en el de Amazon Distribuido de la Semana 6), identifica un campo que, si lo eliminaras hoy, rompería backward compatibility, y explica por qué.

6. ¿Por qué BACKWARD_TRANSITIVE es una garantía más fuerte que BACKWARD simple, y en qué tipo de organización esa diferencia realmente importa en la práctica?

7. Explica, usando el caso de estudio de Stripe, por qué versionar por fecha es una estrategia distinta —pero con el mismo objetivo— que versionar con /v1/ y /v2/ en la URL.